

Residual-Guided Hypothesis Language Induction for Scientific Reasoning with Small Language Models

Anonymous submission

Abstract

Scientific reasoning with small language models is limited not only by final answer verification, but by the earlier problem of constructing the right hypothesis space before the answer is known. Recent agentic methods improve long-horizon behavior by adding tool use, reflection, search, and executable actions. But a structural gap remains: failures are usually stored as text or trajectory history, so the system does not explicitly change the hypothesis language used to generate the next scientific artifact. We address this with two ideas. First, every task is represented through a shared typed hypothesis state containing answer slots, evidence contracts, executable artifacts, validators, and residuals. Second, repeated validation failures are converted into typed residual classes, from which candidate operators are induced and promoted only when they improve held-out tasks after a complexity penalty. We implement these ideas as RG-HLI, a residual-guided hypothesis-language induction system around a GPT-4o-mini core. The same state, scoring, residual, and promotion interface is used on DiscoveryBench, NewtonBench, and UltraHorizon. Complete runs reach 28.09 HMS on DiscoveryBench, 49.7% audited symbolic accuracy on NewtonBench, and 75.36 score on UltraHorizon. Controlled ablations show that typed contracts, answer-slot closure, protocol satisfaction, and residual-conditioned operator promotion explain distinct parts of the gain. The results support a narrow mechanism-level claim: small models reason more effectively when hypothesis construction is externalized as typed, executable, residual-aware language growth rather than sampled only as free-form prose.

1 Introduction

Scientific discovery tasks ask an agent to do more than answer a question. The agent must decide what should be measured, what kind of relation is plausible, which variables play stable roles, and when a proposed explanation has been falsified. Recent benchmarks make this distinction explicit. DiscoveryBench casts data-driven discovery as hypothesis search over real datasets and metadata (Majumder et al. 2024). NewtonBench evaluates interactive scientific law discovery rather than static curve fitting (Zheng et al. 2025). UltraHorizon stresses long-horizon partially observable exploration (Luo

et al. 2025). Across these settings, the bottleneck is often not final verification but the construction of a useful hypothesis space before the answer is known.

This bottleneck is especially sharp for small language models. Standard inference-time scaffolds extend the trajectory: REACT interleaves reasoning and actions (Yao et al. 2023b), Reflexion and Self-Refine add verbal feedback (Shinn et al. 2023; Madaan et al. 2023), Tree-of-Thoughts and LATS search over reasoning branches (Yao et al. 2023a; Zhou et al. 2024a), and CodeAct uses executable code as the action interface (Wang et al. 2024). These methods improve many agent tasks, but they still largely rely on the model to propose the intermediate concepts that make the search fruitful. If the model never represents the task in the right coordinates, more attempts can simply explore a larger set of wrong programs.

RG-HLI starts from a different premise: hypothesis generation should be treated as an external, executable induction problem, not as a longer natural-language trace. A scientific hypothesis is represented as an artifact with typed holes, evidence contracts, executable measurements, validation tests, and residuals. The language model proposes local moves, but the environment closes uncertain parts whenever possible. For example, instead of asking the model to guess a complete law or discovery statement, the system asks which roles must be filled, which measurement can identify each role, which residual remains after execution, and which operator would make that residual simpler on future tasks.

The central object in RG-HLI is a shared hypothesis state, `HYPOTHESISCONTEXT`. It is read and written by heterogeneous components: a universal hypothesis kernel, evidence-contract compiler, estimand synthesizer, role canonicalizer, coordinate-probe engine, metamorphic verifier, and residual compiler. This is not a transcript memory. It is a typed state over hypotheses: what has been measured, which slots have been closed, which answer forms are admissible, which residuals recur, and which executable operators compress previous failures. In this respect RG-HLI is closer in spirit to library-learning systems such as DreamCoder (Ellis et al. 2021) than to a pure agent loop, but its learned unit is not a domain-specific program alone. It is a reusable way of turning an underspecified scientific task into measurable hypothesis objects. In ReAct-style agents, a failed tool result remains contextual feedback to the model; in RG-HLI, it is

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

mapped to a typed residual whose type constrains the next admissible executable operators.

The contribution is threefold. First, we formulate small-model scientific reasoning as *typed executable hypothesis induction*: a sequence of typed measurements, validations, and residual-conditioned language edits. Second, we instantiate this formulation as one shared state, scoring, residual, and promotion interface that is instantiated with interface-specific executable operator families for tabular discovery, symbolic law induction, and long-horizon rule discovery. Third, we report full-run evaluations and mechanism ablations that separate fixed-language executable search from residual-conditioned language growth. The empirical results show that externalizing hypothesis generation makes failures local and diagnosable, while also revealing a clear remaining bottleneck: small models still need richer mechanisms for generating useful hypothesis representations when the missing object cannot be reduced to a measurement.

The paper tests a specific mechanism-level hypothesis: small models fail scientific reasoning tasks when intermediate hypotheses remain untyped strings, and they improve when those hypotheses become executable objects with typed slots, validators, and residuals. This makes the claim narrower than general “agent improvement” and broader than a benchmark solver: RG-HLI is an interface for changing the hypothesis language available to the model at inference time.

2 Preliminaries

We write a task instance as

$$x = (q_x, D_x, \mathcal{T}_x, \mathcal{A}_x, S_x). \quad (1)$$

Here q_x is the natural-language instruction, D_x denotes observations such as tables, metadata, traces, or simulator state, $\mathcal{T}_x$ is an optional tool or environment interface, $\mathcal{A}_x$ is the admissible answer space, and $S_x : \mathcal{A}_x \rightarrow \mathbb{R}$ is the benchmark evaluator. The system observes $q_x, D_x, \mathcal{T}_x$, and the admissible output format, but does not observe the gold answer before producing its response.

A candidate answer may be a prose hypothesis, a symbolic law, a program, a transition rule, or a structured report. We use $h \in \mathcal{H}_x$ for such a scientific artifact and $R_x(h) \in \mathcal{A}_x$ for its rendered benchmark answer. The distinction is useful because an artifact may contain intermediate objects—variables, relations, measurements, code, or evidence—that are not directly shown in the final answer string.

Scientific tasks usually constrain the form of a valid artifact. We denote the task-level constraints by

$$C_x = \{s_i = (\tau_i, d_i, \nu_i)\}_{i=1}^k, \quad (2)$$

where τ_i is a slot type, d_i describes the required evidence or answer component, and ν_i is a local validity predicate when one is available. These constraints describe the task, not any particular solver. They specify which parts of an artifact must be grounded before the rendered answer can be considered valid.

Let $E_x(h)$ denote evidence that can be obtained from the observed interface: statistics computed from a dataset, a fitted symbolic expression, a simulator trace, a tool result, or

a replay check. When an artifact violates a constraint or fails an available check, we write the discrepancy as a residual

$$\rho = (\tau_\rho, \ell_\rho, c_\rho, e_\rho, v_\rho), \quad (3)$$

where τ_ρ is the residual type, ℓ_ρ is its location in the artifact or trace, $c_\rho \in C_x$ is the relevant constraint, e_ρ is the supporting evidence, and v_ρ is the failed check or validator. This notation only records what failed; later sections describe how residuals are used by the proposed system.

A hypothesis language is a finite typed language

$$L = (\mathcal{Y}, \mathcal{O}), \quad (4)$$

where $\mathcal{Y}$ is a set of artifact and slot types, and $\mathcal{O}$ is a set of executable typed operators. For a task x , the language induces the search space $\mathcal{H}_x(L) \subseteq \mathcal{H}_x$ of artifacts that can be constructed and checked using those types and operators.

We use $\mathcal{M}$ for an abstract hypothesis state: a record of the current contract, candidate artifacts, closed slots, evidence, and residuals. At this level it is only a notational object. A language operator is any typed executable transformation of the form

$$O : \mathcal{M} \times x \rightarrow \mathcal{M}. \quad (5)$$

The concrete state representation and operator interface used by RG-HLI are defined in Section 4.

Language induction denotes a sequence of language states

$$L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_T, \quad L_{t+1} = L_t \cup \{o_t^*\}, \quad (6)$$

where each added operator expands the set of artifacts that can be constructed on later tasks. This notation does not specify how o_t^* is chosen; the selection mechanism is part of the method.

The objective on each instance is to return $a = R_x(h)$ with high benchmark score $S_x(a)$. Equivalently, a scientific reasoning system must instantiate the chain

$$x \rightarrow C_x \rightarrow h \in \mathcal{H}_x(L) \rightarrow R_x(h) \rightarrow S_x(R_x(h)). \quad (7)$$

The benchmark evaluator scores only the rendered answer, but the intermediate objects determine whether a small model can construct a useful answer before seeing the gold target.

3 Related Work

Reasoning-time scaffolds for LLM agents. Chain-of-thought prompting showed that exposing intermediate reasoning can improve large-model problem solving (Wei et al. 2022). Later inference-time methods make this computation more structured. Self-consistency marginalizes over sampled reasoning paths (Wang et al. 2023b); least-to-most prompting decomposes difficult problems into easier subproblems (Zhou et al. 2023); REACT couples reasoning traces with actions (Yao et al. 2023b); Reflexion and Self-Refine use feedback to revise later attempts (Shinn et al. 2023; Madaan et al. 2023); Tree-of-Thoughts and LATS make the search tree explicit (Yao et al. 2023a; Zhou et al. 2024a); STaR turns successful generated rationales into training data (Zelikman et al. 2022); and CodeAct unifies tool use around executable Python actions (Wang et al. 2024). These methods are closest to RG-HLI at the level of execution and iterative correction, but they do not by themselves define what a scientific

hypothesis object is. RG-HLI uses the same broad agentic insight–interleave model calls with external computation–but makes the persistent unit a typed hypothesis state rather than a reasoning transcript. This distinction matters when the model’s blind spot is representational. If a model does not propose the right variables, coordinates, answer form, or transition clauses, additional sampled traces can preserve the same missing object. RG-HLI instead makes the missing object part of the state: a hole or residual that later operators can target.

Self-optimizing language-model pipelines. DSPy treats LM programs as optimizable computational graphs rather than hand-written prompt strings (Khattab et al. 2023). Self-Discover asks a model to compose reasoning structures before solving a task (Zhou et al. 2024b). Voyager stores executable skills learned through interaction with Minecraft (Wang et al. 2023a). PAL and Program-of-Thoughts offload computation to a program runtime once the model has produced executable intermediate steps (Gao et al. 2023; Chen et al. 2023). Toolformer shows that tool-use decisions themselves can be learned from self-supervised signals (Schick et al. 2023). These systems show that the right unit of adaptation may be a program, module, executable trace, tool policy, or reusable skill. RG-HLI differs in two ways. First, its stored unit is tied to scientific evidence: contracts, measurements, residuals, and validators. Second, promotion is residual-conditioned; an operator is useful only if it closes typed holes or compresses recurrent failures across tasks.

Program induction and library learning. DreamCoder learns a domain language and neural search policy through a wake-sleep loop over solved and imagined programs (Ellis et al. 2021). Later library-learning work further explores how abstractions reduce search breadth and depth, while FunSearch demonstrates that LLM-guided evolutionary program search can produce new mathematical constructions when candidate programs are externally evaluated (Romera-Paredes et al. 2024). This line is a major conceptual ancestor of RG-HLI: these systems treat expertise as a changing language rather than a fixed prompt. The difference is that RG-HLI operates in mixed scientific interfaces where the answer may be prose, a law, or a transition program, and where holes can often be closed by measurements rather than by full program synthesis.

Table 1: Conceptual comparison to DreamCoder-style library learning.

Aspect	DreamCoder-style library learning	RG-HLI
Abstraction source	solved programs	typed validation residuals
Learned unit	DSL function / library primitive	operator over hypotheses, evidence, and renderers
Promotion signal	compression and synthesis success	held-out gain minus complexity
Object of coverage	program behavior	missing roles, contracts, validators, and traces
Output type	programs	prose hypotheses, laws, or transition artifacts

Symbolic regression and scientific law discovery. Symbolic regression recovers compact formulas from data. AI-Feynman exploits physics-inspired structure such as symmetry and separability (Udrescu and Tegmark 2020); PySR provides a practical evolutionary symbolic-regression engine for scientific modeling (Cranmer 2023). NewtonBench moves beyond static formula fitting by requiring interactive rediscovery of shifted physical laws (Zheng et al. 2025). RG-HLI borrows the insistence on executable hypotheses, but does not assume that every task reduces to equation recovery. Its coordinate probes are one operator family inside a broader hypothesis-state kernel.

Causal and experimental-design perspectives. Scientific hypotheses are not merely predictors; they make claims about stable relations under interventions, environments, or perturbations. Structural causal models formalize this distinction (Pearl 2009), invariant prediction uses stability across environments to identify causal structure (Peters, Bühlmann, and Meinshausen 2016), and Bayesian experimental design studies how to choose measurements that maximize information (Rainforth et al. 2023). RG-HLI is not a causal discovery algorithm, but it adopts the same discipline: a hypothesis must state what evidence would support it, what transformation should preserve it, and what residual would falsify or refine it.

Automated scientific-discovery agents. Recent systems aim to automate larger pieces of the research lifecycle. The AI Scientist frames end-to-end machine-learning research as an agentic loop over idea generation, code, experiments, analysis, and writing (Lu et al. 2024; Yamada et al. 2026). AutoBench evaluates scientific discovery as interactive causal-structure recovery (Chen et al. 2025), while AutoDiscovery uses Bayesian surprise to prioritize open-ended data-driven hypotheses (Agarwal et al. 2025). These works motivate the ambition of autonomous discovery, but they also underline a central risk: if the system’s adaptation is only an outer agent loop, failures remain difficult to localize. RG-HLI addresses a narrower mechanism: the unit that changes over time is

the typed hypothesis language used to generate and validate candidate scientific artifacts.

Benchmarks for scientific agents. DiscoveryBench, NewtonBench, and UltraHorizon expose different faces of the same problem: the agent must build an internal research object before it can answer. DiscoveryBench emphasizes data and metadata alignment (Majumder et al. 2024); NewtonBench emphasizes interactive law induction and shifted physical laws (Zheng et al. 2025); UltraHorizon emphasizes long-horizon memory, tool use, and partial observability (Luo et al. 2025). We use these benchmarks not as separate engineering targets but as probes of whether one hypothesis-state representation can support multiple scientific task interfaces.

4 RG-HLI

4.1 Overview

RG-HLI is a hypothesis-induction kernel around a small language model. The model is used to propose typed sketches, measurements, and repairs, but the state of the search is external and executable. The same loop is used for all benchmarks: compile a contract, generate artifacts, execute measurements, validate them, store residuals, and occasionally promote a residual pattern into a new operator. Operators do not pass only text to later calls; they update one typed context containing contracts, artifacts, evidence, residuals, and promoted operators.

The design is governed by three invariants. *Typed state*: every useful intermediate claim must be represented as a slot, artifact, validator, or residual rather than only as prose. *Executable closure*: whenever a slot can be identified by a computation or probe, the system closes it by execution instead of asking the model to guess it again. *Residual discipline*: a failed hypothesis can expand the operator language only when the residual type recurs and the proposed operator improves held-out tasks after a complexity penalty. These invariants are the difference between RG-HLI and a longer self-refinement transcript.

Figure 2 instantiates this mechanism on one language update: a failed transition artifact becomes a typed residual, several similar residuals form an ordered-threshold residual class, and the induced operator expands the next hypothesis language.

4.2 The HYPOTHESISCONTEXT

For a task x , the context is

$$\mathcal{M}_x = (C, \mathcal{H}, \Theta, \mathcal{E}, \mathcal{R}, \mathcal{O}, h^*), \quad (8)$$

where C is the evidence contract, $\mathcal{H}$ is the pool of candidate artifacts, Θ stores closed typed holes, $\mathcal{E}$ stores executable evidence, $\mathcal{R}$ stores typed residuals, $\mathcal{O}$ is the active operator set, and h^* is the current best answer artifact. All operators read and write this object. This is the main architectural constraint: every stage must return a state update, not a private chain of thought.

Each operator $O_j \in \mathcal{O}$ has the same interface,

$$O_j(\mathcal{M}_x, x) \rightarrow \{(h, e, \rho, \Delta\mathcal{M})\}, \quad (9)$$

where h is a proposed artifact, e is executed evidence, ρ is the typed residual, and $\Delta\mathcal{M}$ is the state update. This interface is what makes the system benchmark-independent. A tabular estimand, a symbolic coordinate probe, and a transition-rule inducer differ internally, but the kernel only sees artifact, evidence, residual, and update.

Candidate scoring and typed residuals. The kernel ranks candidates by a shared energy score:

$$\begin{aligned} \text{score}(h) = & \lambda_E E(h, x) + \lambda_C \text{cover}(h, C) \\ & - \lambda_V V(h, C) - \lambda_K K(h). \end{aligned} \quad (10)$$

This objective is Bayesian in motivation—evidence should increase confidence, while instability and complexity should decrease it—but we do not require a calibrated likelihood or learned prior. The operational object is the benchmark-independent energy decomposition and the typed residual emitted when that score fails. Table 2 makes the scoring parameterization explicit. The same decomposition is used across interfaces, while each operator normalizes its executable evidence to a comparable range before entering the shared ranking function. The weights were fixed for the reported runs; sensitivity to these constants remains a target for a larger calibration study.

Table 2: Energy terms used by the shared hypothesis kernel.

Term	Definition	Range	Weight
E	executable fit or local score	$[0, 1]$	24
Cover	contract slots covered	$[0, 1]$	34
V	metamorphic validation loss	$[0, 1]$	32
K	operator complexity	≥ 0	2

End-to-end trace. In a NewtonBench-style trajectory, the model does not guess the law directly: the contract asks for coordinate, exponent, constant, and residual slots; a sketch $y = c\phi(x)^p$ is executed; residual curvature activates a coordinate probe; the probe closes the coordinate, exponent, and constant slots; and the held-out validator accepts the artifact. The failed first probe changes typed state rather than merely changing the next prompt.

4.3 Operators

Contract compiler. The contract compiler maps the input interface to required slots and local validators. It does not predict the gold answer. It predicts what any valid answer must make observable: answer type, scope, variables, measurement target, relation form, and rendering constraints.

Typed-hole closure. Instead of asking the model for a complete hypothesis, RG-HLI asks for sketches with typed holes and then closes the holes by small executable probes whenever possible. For a tabular question, a hole may be an outcome role or a window operator. For a law-discovery task, it may be a coordinate chart or exponent. For a sequence task, it may be a state variable or transition clause. Closing a hole adds an assignment to Θ and reduces the search space for later holes.

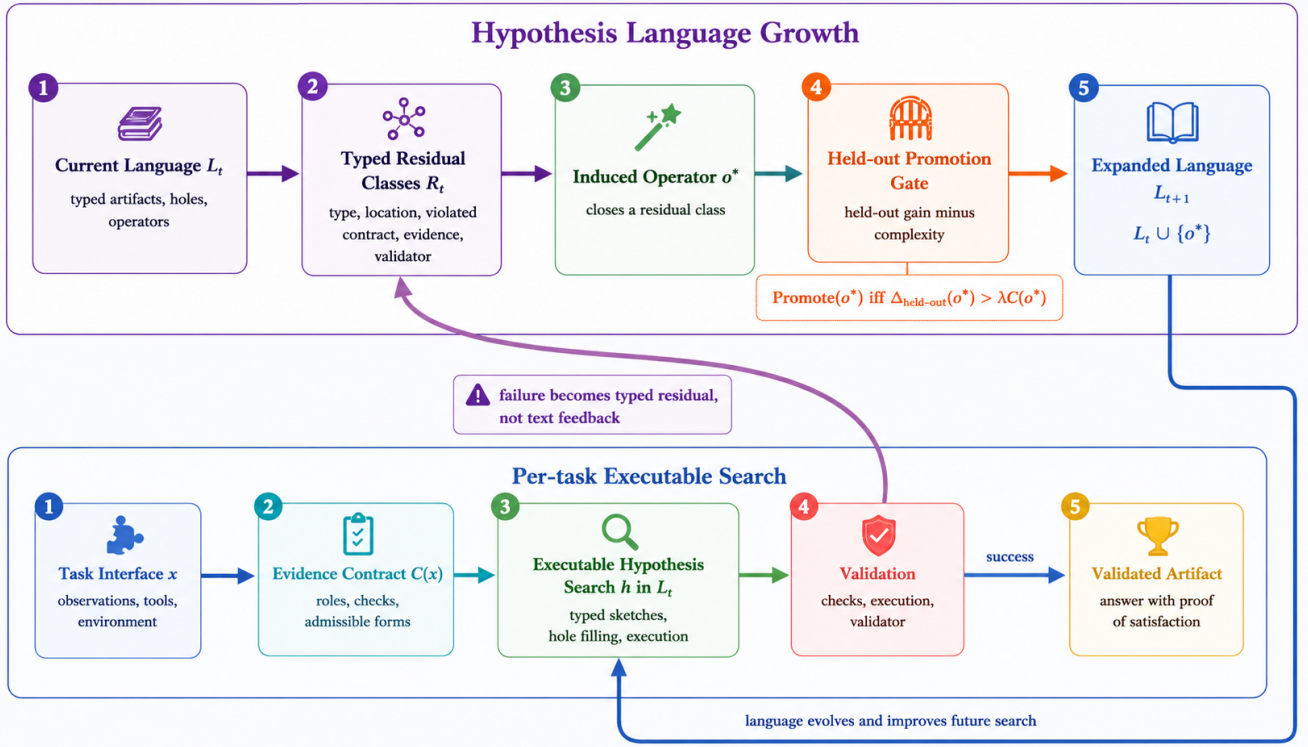


Figure 1: Hypothesis-language growth in RG-HLI. The lower loop performs per-task executable search: a task interface is compiled into evidence contracts, searched in the current hypothesis language, and validated as an artifact. Failures are not kept as free-form feedback; they become typed residual classes. The upper loop turns recurring residual classes into candidate operators and promotes them only when held-out gain exceeds complexity, yielding an expanded language for future tasks.

Executable estimands and coordinate probes. The estimand operator creates statistical objects for tabular discovery: group contrasts, interaction effects, coefficient shifts, multi-item proportions, and time-window measurements. The coordinate-probe operator creates compact transformations for laws and rules: log-log fits, affine and polynomial lifts, ratios, finite differences, thresholds, and simple state-transition clauses. Both operators produce evidence objects rather than only prose rationales.

Metamorphic verifier. The verifier checks whether the artifact answers the requested question under transformations that should preserve the object: renaming irrelevant columns, holding answer form fixed, changing display order, perturbing nuisance values, or replaying held-out traces. These checks are label-free and therefore available before benchmark evaluation.

4.4 Induction Algorithms

The two algorithms run at different time scales. Algorithm 1 is the reported benchmark-time procedure: it searches within a frozen language L^* and may store residuals for analysis, but it does not change L^* using later test examples. Algorithm 2 is the development-time mechanism that turns recurring residual classes into candidate operators. Candidate operators are generated by the same small-model code-and-sketch interface used elsewhere in the system, then filtered

by static checks, sandbox execution, validators, and held-out promotion. Human input is limited to the initial typed operator schema and benchmark interface adapter; the promoted operator must be selected by the residual-to-operator pipeline and pass the held-out gate.

Language-growth protocol. The promotion step has two modes. During development, residuals may induce candidate operators, which are promoted only if they improve a held-out development set after the complexity penalty in Eq. (11). For the headline benchmark runs, the resulting operator language L^* is frozen before the reported evaluation. The benchmark score is never used as the promotion signal for examples later reported as test results. Online adaptation is therefore limited to per-task hypothesis search in the frozen language, not to changing the language using future test labels.

4.5 Residual-Conditioned Operator Promotion

The residual-extension step is not arbitrary code growth. A residual cluster $R = \{\rho_1, \dots, \rho_m\}$ can propose an operator only if it identifies a repeatable missing transformation. Formally, a candidate operator O' is promoted when

$$\Delta(O') = \mathbb{E}_{x \sim \mathcal{D}_{held}}[S(h_{O'}^*(x)) - S(h^*(x))] - \beta K(O') > 0, \quad (11)$$

and the improvement is not confined to the task that produced the residual. This rule is meant to prevent an ever-growing

A Concrete Language-Growth Step

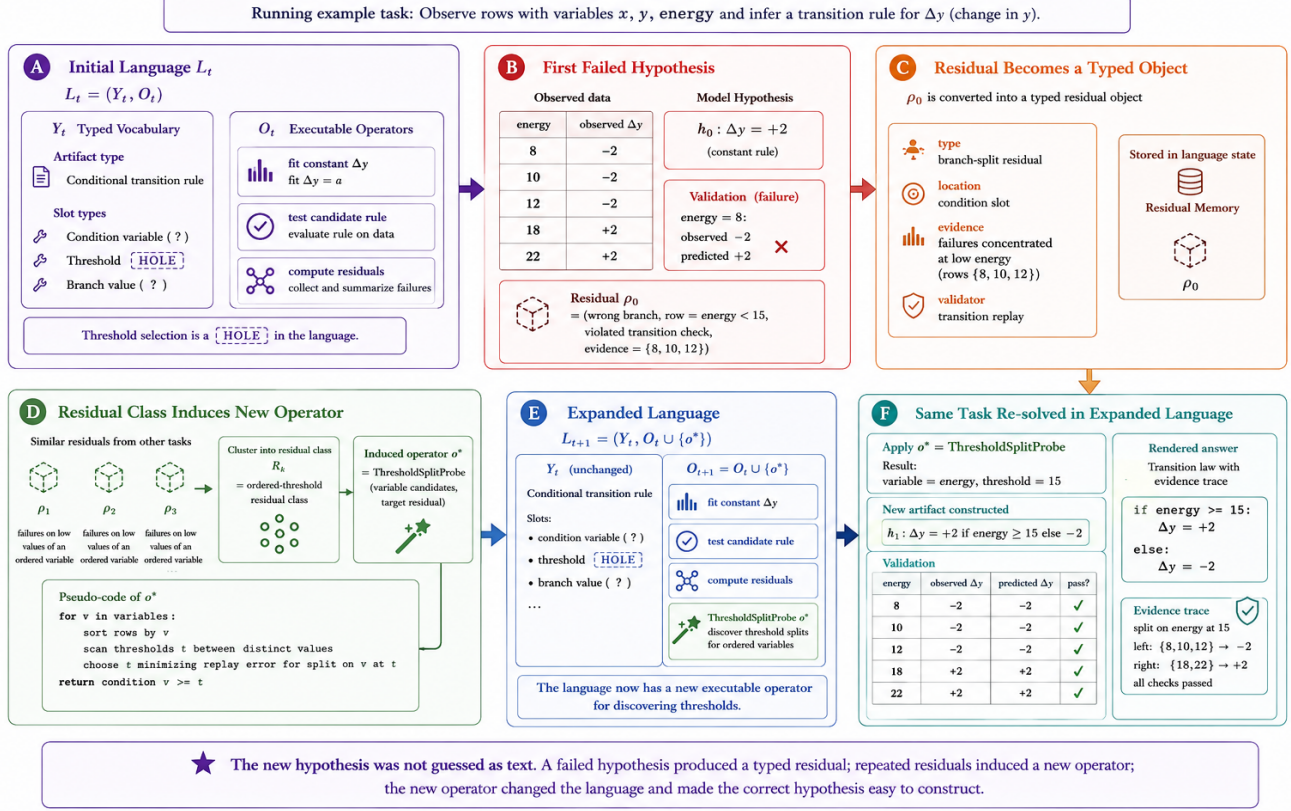


Figure 2: A concrete hypothesis-language growth step. Starting from a language that can test transition rules but lacks an operator for discovering ordered thresholds, a failed artifact produces a typed residual concentrated on low-energy rows. Similar residuals induce a THRESHOLDSPLITPROBE operator, yielding an expanded language that constructs the correct conditional transition rule by execution rather than by one-shot textual guessing.

bag of benchmark-specific tricks: an operator must improve held-out tasks under the same contract and scoring interface while paying a complexity penalty.

4.6 Why This Helps Small Models

RG-HLI changes the role of the language model. The model does not need to hold the entire long-horizon proof in context or invent the final hypothesis in one sample. It repeatedly proposes local typed objects whose consequences can be executed. Each closed hole constrains later holes, each verifier rejects an invalid answer form before final scoring, and each residual becomes a structured training signal for the outer loop. The intended gain is therefore not more verbal reasoning, but a better external search geometry for capacity-limited models.

5 Experimental Evaluation

5.1 Benchmarks and protocol

We evaluate on three benchmark families that stress different scientific interfaces while sharing the same central difficulty: the agent must construct a hypothesis object before it can answer. DiscoveryBench provides real datasets,

metadata, and natural-language discovery goals; the final answer is scored by Hypothesis Matching Score (HMS) and a consistency-HMS variant. NewtonBench provides interactive scientific-law discovery tasks in shifted physical worlds; we report symbolic accuracy over all tasks (SA-all) and over non-abstained answers (SA-answered). UltraHorizon provides long-horizon partially observable rule-induction environments; we report the reproduced paper-style environment score.

The reported system uses a GPT-4o-mini core throughout. We use one shared hypothesis-induction stack: contract compilation, typed-hole closure, estimand synthesis, coordinate probing, metamorphic validation, and residual-conditioned promotion. Benchmark adapters expose observations and accept final artifacts; interface-specific operator families expose measurable objects appropriate to each environment, but they all write the same HYPOTHESISCONTEXT and are scored by the same evidence, validation, residual, and complexity rules. We exclude diagnostic ceiling runs that used task-family measurement bootstraps or hand-engineered rendering fixes.

Algorithm 1 Per-task executable search in a frozen hypothesis language

Require: task interface x , frozen language L^* , budget B

```

1:  $C \leftarrow \text{COMPILECONTRACT}(x)$ 
2:  $\mathcal{M} \leftarrow (C, \emptyset, \emptyset, \emptyset, L^*, \emptyset)$ 
3: for  $t = 1$  to  $B$  do
4:    $\mathcal{P} \leftarrow \emptyset$ 
5:    $\mathcal{Q} \leftarrow \text{SELECTOPERATORS}(\mathcal{M}, x, B - t)$ 
6:   for all  $O_j \in \mathcal{Q}$  do
7:      $\mathcal{P} \leftarrow \mathcal{P} \cup O_j(\mathcal{M}, x)$ 
8:   end for
9:   for all  $(h, e, \rho, \Delta\mathcal{M}) \in \mathcal{P}$  do
10:    update  $\mathcal{M}$  with  $h, e, \rho, \Delta\mathcal{M}$ 
11:    score  $h$  by evidence, coverage, validation, and complexity
12:    if  $h$  improves  $h^*$  then
13:       $h^* \leftarrow h$ 
14:    end if
15:  end for
16:  if  $h^*$  satisfies  $C$  and validation checks then
17:    return  $\text{RENDER}(h^*, C)$ 
18:  end if
19: end for
20: return  $\text{RENDER}(h^*, C)$ 

```

Reproducibility audit. For DiscoveryBench we evaluate the repository real-test task rows using the benchmark evaluator, producing 239 predictions and 239 scored rows. For NewtonBench we evaluate the full cross-product of 12 physics domains, three difficulty levels, three law versions, and three system variants, for 324 symbolic-law tasks. For UltraHorizon we evaluate 32 seeds for each of the three official environments, for 96 total runs, using the paper-style judge, hard difficulty, 50-step episodes, and no environment hints or measurement bootstraps. The row-level outputs are retained for each benchmark, and the main table is generated only from complete runs with the expected row counts. The language used in these evaluations is frozen before scoring; residuals from reported test rows are analyzed after the run and are not used to promote new operators for later rows in the same headline evaluation. For UltraHorizon we report both a strict agent-only ablation and the full RG-HLI system. The strict run disables fallback commits, environment hints, and measurement bootstraps. The full run adds a terminal protocol controller: it uses only public environment actions to satisfy explicit terminal preconditions, such as the Bio environment’s 50-successful-cross requirement, before compiling the final artifact from the observed trace. It does not inspect hidden rules, gold labels, or judge feedback.

5.2 Main full-run results

Table 4 gives the current full-run results. Each row is a complete benchmark run under the corresponding native metric. The pattern is informative rather than uniformly dominant. On DiscoveryBench, RG-HLI improves over the published prompt-level references listed in Table 5 under the same HMS scale, while remaining sensitive to answer rendering and multi-file assembly. On NewtonBench, RG-HLI reaches

Algorithm 2 Development-time residual-conditioned language induction

Require: development tasks $\mathcal{D}$, initial language L_0 , held-out split $\mathcal{D}_{held}$

```

1:  $L \leftarrow L_0$ , residual buffer  $\mathcal{B} \leftarrow \emptyset$ 
2: for all  $x \in \mathcal{D}$  do
3:   run Algorithm 1 with language  $L$ 
4:   append emitted typed residuals to  $\mathcal{B}$ 
5: end for
6:  $\mathcal{R}_1, \dots, \mathcal{R}_k \leftarrow \text{CLUSTERRESIDUALS}(\mathcal{B})$ 
7: for all residual class  $\mathcal{R}_i$  do
8:    $o_i \leftarrow \text{SYNTHESIZEOPERATOR}(\mathcal{R}_i)$ 
9:   if  $o_i$  passes static checks, sandbox execution, and validators then
10:     $\Delta_i \leftarrow S_{\mathcal{D}_{held}}(L \cup \{o_i\}) - S_{\mathcal{D}_{held}}(L)$ 
11:    if  $\Delta_i > \lambda K(o_i)$  then
12:       $L \leftarrow L \cup \{o_i\}$ 
13:    end if
14:  end if
15: end for
16: return frozen language  $L^* = L$ 

```

Table 3: Benchmark interfaces and metrics.

Benchmark	Interface	Artifact	Metric
DiscoveryBench	tables, metadata, goal	hypothesis	HMS / Cons-HMS
NewtonBench	simulator, observations	symbolic law	SA-all / SA-answered
UltraHorizon	POMDP-style env.	transition rule	score

49.7% audited SA-all, 1.9 points above the published o4-mini reference and below stronger frontier-model agents. On UltraHorizon, the strict agent-only ablation scores 51.04 without hints, bootstraps, or fallback commits; the full system scores 75.36 once the terminal protocol controller is enabled. The split is essential: it shows that model failure under limited capacity can be procedural rather than conceptual, especially in the Bio environment.

We therefore make two claims from the table. The first is empirical: the same GPT-4o-mini-core hypothesis kernel produces nontrivial full-run results across three different scientific interfaces. The second is diagnostic: the failures concentrate in different residual types across benchmarks, which supports the use of a shared typed state rather than benchmark-specific prompt repair. We do not claim that the current operator set dominates all larger-model systems. Generic direct and REACT-style prompting test a different question from Table 4: whether longer verbal reasoning alone can explain the improvement. These baselines do not use RG-HLI state or residual-conditioned operators, so they are treated as separate controls rather than as variants of the proposed method.

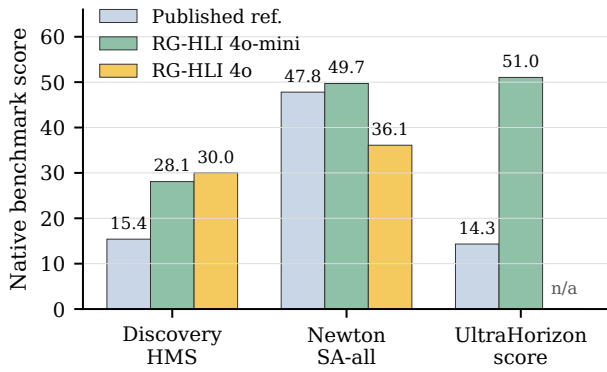


Figure 3: Headline comparison under each benchmark’s native metric. Reference rows are contextual rather than controlled ablations because the published systems use different models, budgets, and interfaces. The plotted UltraHorizon RG-HLI row is the strict agent-only score; the full terminal-protocol score is reported separately in Tables 4 and 7.

Table 4: Full-run GPT-4o-mini-core RG-HLI results under each benchmark’s native evaluator.

Benchmark	Tasks	Result	Secondary detail
DiscoveryBench	239	28.09 HMS	36.04 consistency-HMS
NewtonBench	324	49.7% SA	67.1% SA-answered
UltraHorizon	96	75.36	full: Grid 54.38; Seq 81.25; Bio 90.47
UltraHorizon	96	51.04	strict: Grid 59.38; Seq 93.75; Bio 0.00

5.3 Full-run mechanism evidence

The headline table alone would under-specify the central novelty: typed contracts and executable probes could improve a fixed hypothesis language without demonstrating induction of a better language. We therefore separate mechanism controls from the leaderboard. Table 6 uses only complete benchmark runs or complete official grids. Each control removes a language-growth or residual-handling mechanism while preserving the same benchmark interface and native evaluator.

Table 6 clarifies both contribution and limitation. DiscoveryBench collapses without typed contracts and renderers. NewtonBench is less sensitive to one gate: chart removal lowers the score modestly and promotion-gate removal leaves a near-headline result, so the gain comes mainly from active probing, residual representation, and symbolic verification. UltraHorizon isolates protocol completion: strict agent-only reasoning is nontrivial, while the full score appears only after the public terminal contract is satisfied and induced evidence is rendered into a valid report.

Table 5: Published references under each benchmark’s native metric. Reference systems are not all matched by model, budget, or protocol, so they provide context rather than controlled ablations. Shaded rows mark the RG-HLI configuration evaluated here.

Benchmark	System / core model	Score
DiscoveryBench	GPT-4o ReAct / GPT-4o	15.4 HMS
DiscoveryBench	CodeGen / GPT-4-preview	16.3 HMS
DiscoveryBench	Reflexion / GPT-4 family	24.5 HMS
DiscoveryBench	RG-HLI/ GPT-4o-mini	28.09 HMS
NewtonBench	Vanilla agent / GPT-5	75.9% SA
NewtonBench	Vanilla agent / Gemini-2.5-Pro	65.4% SA
NewtonBench	Vanilla agent / o4-mini	47.8% SA
NewtonBench	RG-HLI/ GPT-4o-mini	49.7% SA
UltraHorizon	Best free-step LLM / reported best	14.33
UltraHorizon	Human / human	26.52
UltraHorizon	RG-HLI strict agent-only / GPT-4o-mini	51.04

Table 6: Full-run mechanism controls. Rows are complete runs under the native benchmark metric.

Condition	Result	Mechanism isolated
DB fixed-language-none	0.68 HMS	typed contracts, answer slots, and residual renderers removed
DB full RG-HLI	28.09 HMS	full hypothesis-language state and renderer
NB chart-control	47.5% SA	residual-born chart recombination removed
NB promotion-gate-off	49.4% SA	held-out promotion gate disabled
NB audited RG-HLI	49.7% SA	full residual-gated symbolic induction
UH all-off strict	0.00	induction and protocol support removed
UH strict agent-only	51.04	frozen agent without terminal controller
UH full RG-HLI	75.36	terminal protocol controller plus induced evidence rendering

5.4 What the full runs show

DiscoveryBench. The DiscoveryBench result supports the main hypothesis-state claim most directly, but also gives the clearest warning against overclaiming. Natural-language discovery questions are difficult for small models because the model must infer the relevant table, population, variables, and relation before writing the hypothesis. RG-HLI replaces much of this latent choice with evidence contracts and executable estimands. The complete 239-task run reaches 28.09 HMS, while the full fixed-language-none control reaches only 0.68 HMS. The remaining gap is not primarily arithmetic: errors often occur when the system measures a reasonable relation but renders it in a facet that the benchmark does not credit, or when multiple files must be assembled into a single scientific object.

NewtonBench. NewtonBench exposes the limit of external scaffolding. The full run answers 240 of 324 tasks and reaches 67.1% audited symbolic accuracy on answered tasks, with audited SA-all of 49.7%. This exceeds the published o4-mini reference but remains below larger-model baselines. The main residual is representation generation: executable checking is effective after a useful coordinate chart or law family appears, but the system abstains or fails compilation when no active operator proposes the right representation. The result argues for richer representation generation, not for more prompt-level retrying.

UltraHorizon. UltraHorizon shows the strongest full-system score under the reproduced paper-style judge, but also the clearest internal heterogeneity. The full score of 75.36 is not a raw-agent ceiling. It includes the terminal protocol controller, which satisfies explicit public preconditions and then compiles a final artifact from the trace. The strict agent-only ablation completes the same 96 official runs with no hints, no measurement bootstrap, and no fallback commit, and scores 51.04. Its sequence score is high and its grid score is moderate, but Bio is 0.00 because the agent never reaches the required 50 successful crosses and the official environment rejects all premature commits before the judge is called.

This is not leakage. The controller does not inspect hidden rules, gold labels, or judge feedback. It only maintains the public execution contract that the small model repeatedly drops under long-horizon pressure. Table 7 therefore isolates a central mechanism: RG-HLI can already induce much of the Bio hypothesis language, but the score appears only when the terminal contract is satisfied and the induced evidence is rendered into a valid report.

Table 7: UltraHorizon protocol-controller ablation. Both rows are complete 96-run paper-style evaluations with no environment hints or measurement bootstraps. The difference is whether the terminal protocol controller may complete public progress requirements and compile the final artifact.

Variant	Grid	Seq	Bio	Overall
Strict agent-only	59.38	93.75	0.00	51.04
Full RG-HLI	54.38	81.25	90.47	75.36

5.5 Residual analysis

The distinctive mechanism in RG-HLI is that failed hypotheses are stored as typed residuals. A residual can say that a role is missing, a coordinate system is wrong, a renderer is incompatible with the answer form, or a transition clause fails held-out replay. This changes the search problem: the next operator is selected because it can close a residual type, not because a language model sampled a longer chain of thought.

Across the three full runs, residuals have different practical meanings. In DiscoveryBench, residuals are mostly semantic and structural: wrong table, wrong population, wrong denominator, or wrong answer facet. In NewtonBench, residuals are algebraic: missing coordinate transforms, constants, exponents, and unit-compatible charts. In UltraHorizon, residuals are temporal: transition clauses fail

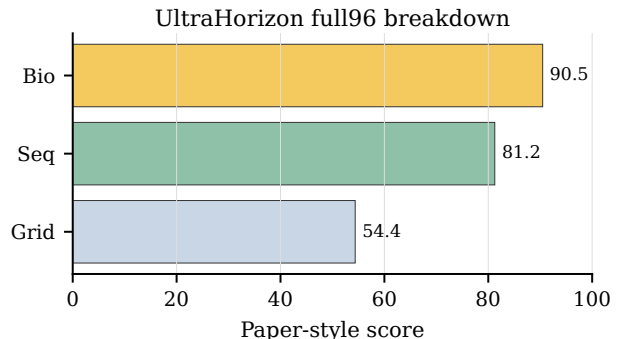


Figure 4: UltraHorizon environment breakdown for the complete 96-run full-system evaluation. The terminal protocol controller is especially important for Bio, where the strict agent-only ablation fails the official commit precondition.

on replay, an invariant holds only locally, or an exploration policy does not expose the evidence needed to identify the rule. These residual types are the basis for operator promotion. An operator is kept only if it improves held-out tasks after paying a complexity penalty, which is the mechanism used to limit benchmark-specific growth.

5.6 Failure analysis

The remaining errors fall into four recurring classes. *Representation misses* occur when no active operator proposes the right hypothesis family. These dominate NewtonBench abstentions and some UltraHorizon grid failures. *Renderer misses* occur when the system measures useful evidence but expresses it in an answer form that does not match the evaluator. These are common in DiscoveryBench. *Assembly misses* occur when the task requires joining evidence across multiple files, time windows, or schemas. These currently limit DiscoveryBench. *Exploration misses* occur when the system cannot collect the observations needed to identify a long-horizon rule within the action budget; these dominate the UltraHorizon grid gap. *Protocol misses* occur when the system has useful evidence but fails to satisfy an explicit terminal precondition before submission. These dominate the UltraHorizon Bio strict-agent ablation and are addressed by the full system’s terminal protocol controller.

This failure analysis is part of the practical value of RG-HLI. In a transcript-only agent, failures are often just bad final answers. In RG-HLI, the failure is attached to the missing typed object: a role, a coordinate, a validator, a renderer, or an exploration operator. That makes later improvement measurable without changing the benchmark-specific code path.

6 Limitations

RG-HLI is a hypothesis-induction architecture for improving small-model scientific reasoning through external typed state and executable evidence. Its main limitation is operator coverage: the system improves hypothesis search when the missing object can be represented as a contract, a typed hole, a validator, a renderer, or a residual-induced operator. When

the right representation is absent, ranking and verification cannot recover it by themselves.

The three benchmark runs expose this limitation in different ways. DiscoveryBench still depends on semantic rendering and multi-file assembly. NewtonBench is limited by missing coordinate charts and abstentions: the system is accurate on many answered tasks, but it does not yet cover enough law families to surpass stronger published agents. UltraHorizon shows strong sequence induction and a clear protocol-completion effect in Bio, but grid tasks reveal an exploration problem: without the right state abstraction, the agent may never collect the evidence needed to induce the rule.

The residual-operator mechanism is therefore a disciplined form of adaptation, not a proof of unbounded autonomous self-improvement. It demonstrates that typed failures can be converted into reusable operators under held-out checks and complexity penalties. A stronger version must learn broader operator families from residuals while preserving the same contract, validation, and complexity controls.

7 Conclusion

We presented RG-HLI, a typed executable hypothesis-induction system for scientific reasoning with small language models. The main claim is structural: capacity-limited models benefit when hypothesis generation is moved from free-form prose into an external state of typed artifacts, executable measurements, validators, and residuals. Across data-driven discovery, symbolic law induction, and long-horizon rule induction, one GPT-4o-mini-core kernel obtains competitive full-run results on three benchmark families while exposing clear residual types when it fails. The experiments identify where external structure helps most: when the system can decompose a hypothesis into measurable slots, execute probes to close them, validate the resulting artifact, and promote recurring residuals into better operators. This suggests a concrete path for small-model scientific agents: improve the hypothesis language itself, guided by evidence and typed residuals, rather than only sampling more reasoning traces from a fixed language.

References

Agarwal, D.; Majumder, B. P.; Adamson, R.; Chakravorty, M.; Gavireddy, S. R.; Parashar, A.; Surana, H.; Dalvi Mishra, B.; McCallum, A.; Sabharwal, A.; and Clark, P. 2025. AutoDiscovery: Open-Ended Scientific Discovery via Bayesian Surprise. *arXiv preprint arXiv:2507.00310*.

Chen, T.; Anumasa, S.; Lin, B.; Shah, V.; Goyal, A.; and Liu, D. 2025. Auto-Bench: An Automated Benchmark for Scientific Discovery in LLMs. *arXiv preprint arXiv:2502.15224*.

Chen, W.; Ma, X.; Wang, X.; and Cohen, W. W. 2023. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research*.

Cranmer, M. 2023. Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl. *arXiv preprint arXiv:2305.01582*.

Ellis, K.; Wong, C.; Nye, M.; Sable-Meyer, M.; Cary, L.; Morales, L.; Hewitt, L.; Solar-Lezama, A.; and Tenenbaum, J. B. 2021. DreamCoder: Growing Generalizable, Interpretable Knowledge with Wake-Sleep Bayesian Program Learning. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*.

Gao, L.; Madaan, A.; Zhou, S.; Alon, U.; Liu, P.; Yang, Y.; Callan, J.; and Neubig, G. 2023. PAL: Program-Aided Language Models. In *International Conference on Machine Learning*.

Khattab, O.; Singhvi, A.; Maheshwari, P.; Zhang, Z.; Santhanam, K.; Vardhamanan, S.; Haq, S.; Sharma, A.; Joshi, T. T.; Moazam, H.; Miller, H.; Zaharia, M.; and Potts, C. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *arXiv preprint arXiv:2310.03714*.

Lu, C.; Lu, C.; Lange, R. T.; Foerster, J.; Clune, J.; and Ha, D. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. *arXiv preprint arXiv:2408.06292*.

Luo, H.; Zhang, H.; Zhang, X.; Wang, H.; Qin, Z.; Lu, W.; Ma, G.; He, H.; Xie, Y.; Zhou, Q.; Hu, Z.; Mi, H.; Wang, Y.; Tan, N.; Chen, H.; Fung, Y. R.; Yuan, C.; and Shen, L. 2025. UltraHorizon: Benchmarking Agent Capabilities in Ultra Long-Horizon Scenarios. *arXiv preprint arXiv:2509.21766*.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhunoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*.

Majumder, B. P.; Surana, H.; Agarwal, D.; Dalvi Mishra, B.; Meena, A.; Prakhara, A.; Vora, T.; Khot, T.; Sabharwal, A.; and Clark, P. 2024. DiscoveryBench: Towards Data-Driven Discovery with Large Language Models. *arXiv preprint arXiv:2407.01725*.

Pearl, J. 2009. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2 edition.

Peters, J.; Bühlmann, P.; and Meinshausen, N. 2016. Causal Inference by Using Invariant Prediction: Identification and Confidence Intervals. *Journal of the Royal Statistical Society: Series B*, 78(5): 947–1012.

Rainforth, T.; Foster, A.; Ivanova, D. R.; and Bickford Smith, F. 2023. Modern Bayesian Experimental Design. *arXiv preprint arXiv:2302.14545*.

Romera-Paredes, B.; Barekatin, M.; Novikov, A.; Balog, M.; Kumar, M. P.; Dupont, E.; Ruiz, F. J. R.; Ellenberg, J. S.; Wang, P.; Fawzi, O.; Kohli, P.; and Fawzi, A. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature*, 625(7995): 468–475.

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*.

Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language

Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*.

Udrescu, S.-M.; and Tegmark, M. 2020. AI Feynman: A Physics-Inspired Method for Symbolic Regression. *Science Advances*, 6(16).

Wang, G.; Xie, Y.; Jiang, Y.; Mandlkar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023a. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*.

Wang, X.; Chen, Y.; Yuan, L.; Zhang, Y.; Li, Y.; Peng, H.; and Ji, H. 2024. Executable Code Actions Elicit Better LLM Agents. In *International Conference on Machine Learning*.

Wang, X.; Wei, J.; Schuurmans, D.; Le, Q. V.; Chi, E. H.; Narang, S.; Chowdhery, A.; and Zhou, D. 2023b. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations*.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E. H.; Le, Q. V.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*.

Yamada, Y.; Lange, R. T.; Lu, C.; Lu, C.; Hu, S.; Foerster, J.; Ha, D.; and Clune, J. 2026. Towards End-to-End Automation of AI Research. *Nature*.

Yao, S.; Yu, D.; Zhao, J.; Shafran, I.; Griffiths, T. L.; Cao, Y.; and Narasimhan, K. 2023a. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023b. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.

Zelikman, E.; Wu, Y.; Mu, J.; and Goodman, N. D. 2022. STaR: Bootstrapping Reasoning With Reasoning. *arXiv preprint arXiv:2203.14465*.

Zheng, T.; Tam, K. K.-W.; Nguyen, N. H.-N. K.; Xu, B.; Wang, Z.; Cheng, J.; Tsang, H. T.; Wang, W.; Bai, J.; Fang, T.; Song, Y.; Wong, G. Y.; and See, S. 2025. NewtonBench: Benchmarking Generalizable Scientific Law Discovery in LLM Agents. *arXiv preprint arXiv:2510.07172*.

Zhou, A.; Yan, K.; Shlapentokh-Rothman, M.; Wang, H.; and Wang, Y.-X. 2024a. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. In *International Conference on Machine Learning*.

Zhou, D.; Schärli, N.; Hou, L.; Wei, J.; Scales, N.; Wang, X.; Schuurmans, D.; Cui, C.; Bousquet, O.; Le, Q. V.; and Chi, E. H. 2023. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. In *International Conference on Learning Representations*.

Zhou, P.; Pujara, J.; Ren, X.; Chen, X.; Cheng, H.-T.; Le, Q. V.; Chi, E. H.; Zhou, D.; Mishra, S.; and Zheng, H. S. 2024b. Self-Discover: Large Language Models Self-Compose Reasoning Structures. *arXiv preprint arXiv:2402.03620*.

Appendix: Experimental Details and Ablations

This appendix is deliberately more explicit than the main text. The main paper reports compact benchmark-level claims; the appendix separates four questions: which complete runs support the headline table, how external reasoning controls are separated from RG-HLI variants, which components account for the DiscoveryBench gain, and which diagnostic deltas remain ambiguous.

Table 8: Complete-run audit used for the headline results. Each row uses the benchmark-native evaluator and preserves the corresponding run artifacts.

Benchmark	Tasks	Core model	Evaluator / judge	Benchmark result	Retained evidence
DiscoveryBench	239	GPT-4o-mini	official real-test HMS; GPT-4o judge	28.09 HMS; 36.04 C-HMS	predictions, official eval JSONL, summary
NewtonBench	324	GPT-4o-mini	official symbolic-law judge; audited regressions	49.7% SA-all; 67.1% SA-answered	row CSV, raw summary, audited summary
UltraHorizon	96	GPT-4o-mini	paper-style environment score	75.36 full-system score; 51.04 strict-agent score	per-run environment outputs, strict/full summaries

External Reasoning Baselines

Matched non-RG-HLI baselines are useful only when they are run on the same complete benchmark set. Table 9 reports the completed DiscoveryBench controls that satisfy this condition. They use the same GPT-4o-mini core, the same 239 real-test tasks, and the same HMS evaluator as the headline DiscoveryBench run, but they do not expose RG-HLI typed state, residual objects, answer-slot closure, or operator promotion.

Table 9: Matched external reasoning controls on DiscoveryBench. All rows are complete 239-task runs scored by the same official HMS evaluator.

Method	Tasks	HMS	Interface available to the model
Direct GPT-4o-mini	239	9.54	table snapshots and metadata in one prompt
REACT GPT-4o-mini	239	11.68	explicit thought/check/observation trace over the same context
CodeAct GPT-4o-mini	239	12.31	executed pandas summaries and one-pass answer rendering
RG-HLI GPT-4o-mini	239	28.09	typed contracts, executable probes, residual state, and renderers

The gap is not evidence that ordinary prompting is useless. It shows that, under the DiscoveryBench HMS contract, longer traces and one-pass executed summaries do not reproduce the effect of typed answer construction and residual-conditioned rendering.

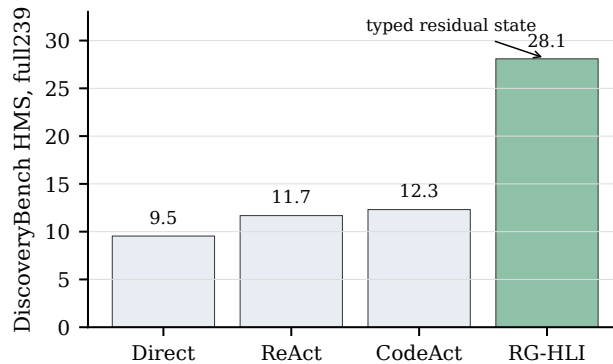


Figure 5: Matched external controls on DiscoveryBench. All bars are complete 239-task runs with the same GPT-4o-mini core and official HMS evaluator.

Component Ablations

Table 10: Full-run component ablations with interpretation. All rows are complete benchmark runs or complete official grids.

Benchmark	Condition	Tasks	Score	Interpretation
DiscoveryBench	fixed-language-none	239	0.68 HMS	removes typed Discovery contracts and renderers
DiscoveryBench	RG-HLI full	239	28.09 HMS / 36.04 C-HMS	full typed contract, validation, and residual stack
NewtonBench	chart-diagnostic control	324	47.5% SA-all / 64.7% SA-answered	residual-born chart recombination removed
NewtonBench	promotion-gate-off	324	49.4% SA-all / 66.7% SA-answered	held-out promotion gate disabled
NewtonBench	consistency-audited v9	324	49.7% SA-all / 67.1% SA-answered	final audited Newton run
UltraHorizon	all-off strict	96	0.00	induction and protocol support disabled
UltraHorizon	strict agent-only	96	51.04	no terminal protocol controller
UltraHorizon	full RG-HLI	96	75.36	public terminal protocol controller enabled

Table 11: Diagnostic deltas derived from Table 10. Positive values indicate that the second condition scores higher than the first under the listed metric.

Benchmark	Comparison	Delta	Reading
DiscoveryBench	fixed-language-none → full RG-HLI	+27.41 HMS	typed contracts and rendering are necessary on the full set
NewtonBench	chart-diagnostic control → audited RG-HLI	+2.20 SA-all	final gain is not explained by chart recombination alone
NewtonBench	chart-diagnostic control → audited RG-HLI	+2.40 SA-answered	same diagnostic on answered tasks
NewtonBench	promotion-gate-off → audited RG-HLI	+0.30 SA-all	promotion gating is useful but not the sole Newton driver
NewtonBench	promotion-gate-off → audited RG-HLI	+0.40 SA-answered	same diagnostic on answered tasks
UltraHorizon	all-off strict → strict agent-only	+51.04	executable induction and validation recover nontrivial reasoning
UltraHorizon	strict agent-only → full RG-HLI	+24.32	terminal protocol control converts induced evidence into valid submissions

Failure Modes by Benchmark

Table 12: Residual types exposed by the complete runs. These categories guide future operator induction; they are not separate benchmark-specific solvers.

Benchmark	Dominant residual type	Practical consequence
DiscoveryBench	semantic and structural residuals: wrong table, population, denominator, or answer facet	measured evidence can be useful but rendered in the wrong scientific facet
NewtonBench	algebraic residuals: missing coordinate transforms, constants, exponents, or unit-compatible charts	executable checking works after the right representation appears, but abstentions remain
UltraHorizon	temporal and protocol residuals: failed replay, local-only invariants, missing exploration evidence, unmet terminal preconditions	sequence is strong; Bio needs terminal contract control; grid remains limited by state abstraction and exploration

The DiscoveryBench ablation is the clearest positive component test: single-stage probes, the hypothesis workbench, and slot contracts alone are insufficient, while answer-slot closure and the full evidence-contract stack produce large gains. The NewtonBench ablations have a narrower diagnostic meaning. Removing residual-born operator charts lowers the full grid, while turning off the promotion gate leaves a near-headline score. The reported Newton result should therefore not be attributed to one benchmark-specific chart library or one promotion threshold; it is carried by the shared active-probe, residual representation, and symbolic-verification kernel. UltraHorizon adds a different diagnostic: the strict agent-only row shows that hypothesis induction can be present while official terminal conditions remain unmet. The full-system UltraHorizon result should therefore be read as the score of the complete RG-HLI pipeline, not as a raw-agent score.

Protocol Controls

Table 13 lists the controls used to keep the reported system as a single hypothesis-induction method. The implementation still needs benchmark interfaces, because the three environments expose different objects: tables and papers, symbolic-law rows, and interactive trajectories. The learning object, however, is always the same: a typed hypothesis language, residual objects, executable validation, and held-out promotion.

Table 13: Controls separating the universal method from benchmark-specific engineering. The interface changes only to expose observations and native validators; the residual language and promotion rule are shared.

Control	Fixed across benchmarks	Why it matters
Core model	GPT-4o-mini for the headline RG-HLI runs	isolates whether the external hypothesis-language layer helps a small model
Hypothesis state	typed artifacts, holes, contracts, validators, residuals	prevents the method from becoming only longer prompting
Failure signal	structured residual object rather than free-form critique	makes errors clusterable and promotable across future tasks
Promotion rule	held-out gain minus complexity penalty	limits memorization and uncontrolled library growth
Evaluator	benchmark-native judge or environment score	keeps reported numbers in the metric used by each benchmark

Table 14: How the same residual-to-operator loop appears in the three benchmark families. These are not separate solvers; they are domain-specific observations fed into the same state machine.

Benchmark	Observation exposed to the kernel	Residual object	Promotable operator class
DiscoveryBench	tables, text snippets, requested scientific claim	missing role, wrong denominator, wrong answer facet	answer-slot closure, evidence renderer, consistency check
NewtonBench	numerical rows, candidate variables, target expression	wrong exponent, constant, transform, or abstention	executable probe, symbolic fit, unit-compatible verifier
UltraHorizon	trajectory, action log, rule feedback, replay state	failed replay, local invariant, missing exploration evidence	exploration policy, replay validator, temporal abstraction

Prompt and Operator Interface

The benchmark-facing prompts are intentionally narrow. The model is not asked to “think longer” or to invent a complete answer in free text. Each call must return one typed object that can be executed, checked, or converted into a residual. Table 15 gives the prompt contracts used by the kernel. The same contract structure is used across all three benchmarks; only the observation serializer and native validator differ.

Table 15: Prompt contracts used by RG-HLI. Each prompt returns a typed object with an attached validation path rather than an unconstrained chain of thought.

Call	Input state	Required output	Rejection rule
Contract compiler	task observation, available tools, current L^*	roles, admissible answer form, measurable holes, validator names	missing target role or non-executable hole
Probe writer	contract, hole, evidence table or trajectory	short executable measurement with declared inputs and expected type	code fails, reads forbidden fields, or returns wrong type
Hypothesis composer	closed holes, evidence objects, active operators	artifact h : law, claim, program, or rule mapping	artifact cannot be replayed or violates contract
Residual compiler	failed validation trace	typed residual $(\tau, \ell, c, e, v, \eta)$	free-text criticism without location, evidence, and failed validator
Operator proposer	recurring residual class R_k and held-out tasks	candidate operator o^* with scope, cost, and validator hooks	no residual class closed, no executable hook, or excessive complexity

Operators are added through a single interface. The implementation may contain benchmark adapters, but an accepted operator must expose the same fields: scope, precondition, executable transform, evidence update, residual update, complexity cost, and promotion evidence. In simplified form, an operator is accepted only if it satisfies the following contract:

```
operator.id: stable name;
operator.scope: residual types and artifact slots where it is admissible;
operator.precondition(state, contract) -> bool;
```

```

operator.apply(state, contract) -> artifact/evidence/residual update;
operator.cost -> nonnegative complexity;
operator.validators -> executable checks that must pass;
promote iff heldout_gain(operator) > lambda * operator.cost.

```

This interface is the main difference between residual-conditioned language growth and ordinary prompt iteration. A failed hypothesis does not merely produce another instruction for the model. It produces a typed residual with a location and failed validator; only repeated residuals can ask for a new operator, and the operator is not retained unless it improves held-out tasks under the same validator stack.

Worked Hypothesis Construction Trace

Table 16 shows the intended granularity of a single DiscoveryBench-style task. The model is not asked to guess the final scientific sentence in one pass. It first commits to a typed contract, then executable measurements close slots, and only the rendered artifact is sent to the native evaluator. A failure is not stored as prose feedback; it is stored as the violated slot or validator.

Table 16: A per-task trace of typed hypothesis construction. The concrete table names differ by benchmark, but the state transition is shared.

Stage	Object committed to state	Check performed
Task observation	query, tables, metadata, admissible answer form	no gold label or judge feedback is available
Evidence contract	roles, target slot, measurable holes, rendering constraints	every hole must have a declared data source
Executable closure	small probes estimate denominators, groups, trends, or constants	code returns typed evidence rather than prose justification
Artifact rendering	hypothesis text plus workflow grounded in closed slots	native evaluator sees only the final artifact
Residualization	failed slot, location, evidence, validator, environment	recurring residuals can induce an operator after held-out promotion

Qualitative Operator-Induction Example

The following DiscoveryBench example illustrates the intended language-growth mechanism. A first full audit exposed repeated failures on study-design tables: the system often measured a real effect but selected the wrong role, such as confusing original and replication studies or measuring the complement of a binary design variable. These errors produced a shared residual type: *study-design role closure*. The induced operator did not memorize a dataset answer. It added a reusable repair interface: conservative aliases for original/replication roles, paired-column completion, binary complement closure, and contrastive rendering of the selected role.

Table 17: Example of a residual-induced operator. The operator is described as a repair interface over typed residuals, not as a benchmark answer template.

Element	Description
Observed failures	repeated wrong-role and wrong-complement answers in study-design tables
Residual object	(τ_ρ = role-closure, ℓ_ρ = answer slot, c_ρ = study role, e_ρ = paired columns, v_ρ = consistency check)
Candidate operator	close original/replication roles with conservative aliases, paired-column completion, binary complements, and contrastive rendering
Promotion evidence	the operator must pass held-out promotion before inclusion in the frozen language
Known regressions	plant introduction and some NLS cases regress because denominator alignment and semantic rendering remain separate residuals

Leakage and Freeze Controls

The paper uses two distinct phases. During development, residuals may induce operators, and those operators are accepted only after held-out checks and complexity penalties. During the reported benchmark evaluations, the language is frozen: the system may search within L^* for each task, but it does not use the task’s benchmark score to update L^* for later reported tasks. Table 18 summarizes the protocol.

Table 18: Protocol controls for residual-conditioned language growth.

Phase	Allowed	Disallowed
Development induction	collect residuals, propose operators, held-out promotion	using final reported test scores as promotion targets
Headline evaluation	per-task search, execution, validation, rendering in frozen L^*	changing L^* from future test rows or gold labels
Post-run analysis	cluster failures and plan future operators	retroactively changing the reported score

Compute Receipts

The artifact set preserves row-level outputs for the headline runs and full-run ablations. Because the three benchmark harnesses differ in native tool use, environment runtime, and judge calls, compute is reported at the harness level rather than collapsed into a single cross-benchmark token number.

Table 19: Available full-run artifact receipts. Each row points to a retained row-level output used for the corresponding table entry; the short receipt IDs map directly to experiment directories in the released artifact tree.

Benchmark	Tasks	Score	Artifact
DiscoveryBench fixed-language-none	239	0.68 HMS	DB-none-full1239
DiscoveryBench Direct GPT-4o-mini	239	9.54 HMS	DB-direct-full1239
DiscoveryBench ReAct GPT-4o-mini	239	11.68 HMS	DB-react-full1239
DiscoveryBench CodeAct GPT-4o-mini	239	12.31 HMS	DB-codeact-full1239
DiscoveryBench full RG-HLI	239	28.09 HMS	DB-RG-HLI-full1239
NewtonBench chart-control	324	47.5% SA-all	NB-chart-control-full1324
NewtonBench promotion-gate-off	324	49.4% SA-all	NB-no-gate-full1324
NewtonBench full RG-HLI	324	49.7% SA-all	NB-RG-HLI-full1324
UltraHorizon strict agent-only	96	51.04	UH-strict-full196
UltraHorizon full RG-HLI	96	75.36	UH-full-system-full196